DAY 5 · LEARN WITH PALLA · STUDY & PRACTICE GUIDE

Type Conversion & Maths with null/undefined

Recap of the Day 5 videos: converting values between types on purpose (and how JavaScript converts them behind your back), plus the two special cases every tester must know — null becomes 0, undefined becomes NaN.

Explicit type conversion

Explicit conversion is when **you** change the type, using the three built-in converter functions. This is the safe, readable way — say what you mean.

```
Number("45")     // 45      — string to number
Number("45px")   // NaN     — not a clean number
Number("")       // 0       — surprise! empty string is 0
String(45)       // "45"    — number to string
String(true)     // "true"
Boolean("hi")    // true    — same rules as truthy/falsy (Day 4)
Boolean(0)       // false
```

- In automation this matters constantly: everything you read from a web page — input values, prices, counts — arrives as a **string**. Convert before you compare: `Number(priceText) > 100`.

Implicit conversion (coercion)

Implicit conversion is when **JavaScript** changes the type for you, based on the operator. The **+** operator is the troublemaker: if either side is a string, it glues (concatenates) instead of adding.

```
"45" + 1     // "451"  — + with a string concatenates!
"45" - 1     // 44     — -, *, / convert to numbers
"45" * 2     // 90
"6" / "2"    // 3
// the classic interview line:
1 + "2" + 3  // "123"
```

null and undefined in maths

When null and undefined land in a numeric expression they behave completely differently — this is a favourite interview question and a real source of test bugs when data is missing.

```
10 + null        // 10   — null converts to 0
10 * null        // 0
10 + undefined   // NaN  — undefined converts to NaN
10 * undefined   // NaN
Number(null)       // 0
Number(undefined)  // NaN
// NaN poisons everything it touches:
NaN === NaN        // false!  use Number.isNaN(x)
```

- Rule of thumb: **null** → **0**, **undefined** → **NaN** — and any calculation containing NaN stays NaN.

Quick reference

Expression	Result — and why
`Number("45") / Number("")`	45 / 0 — strings convert cleanly; empty string is 0
`Number("45px")`	NaN — not a valid number
`"45" + 1`	"451" — + with a string concatenates
`"45" - 1`	44 — the other maths operators convert to numbers
`10 + null`	10 — null becomes 0 in maths
`10 + undefined`	NaN — undefined becomes NaN
`NaN === NaN`	false — check with Number.isNaN(x) instead

Practice exercises

Do these in VS Code, run them with Node, and bring questions to the next live Q&A.

1. Predict each result, then verify in Node: `Number("100")`, `Number("100rs")`, `Number("")`, `String(2026)`, `Boolean("false")`.

   Hint: Careful with the last one — "false" is a non-empty string.

2. Predict, then verify: `"5" + 5`, `"5" - 5`, `"5" * "2"`, `1 + "2" + 3`.

3. Predict, then verify: `7 + null`, `7 - null`, `7 + undefined`, `null + undefined`.

   Hint: Remember: null → 0, undefined → NaN.

4. A web page gives you `const price = "1499"` (a string). Write code that adds 18% tax and prints the total as a number.

   Hint: Convert first with Number(price), then multiply by 1.18.

5. Write a function `safeAdd(a, b)` that treats null OR undefined arguments as 0, so `safeAdd(10, undefined)` returns 10 instead of NaN.

   Hint: Convert each argument with (x || 0), or check Number.isNaN after converting.

Bonus challenge — Build `toNumberReport(value)` that prints `` `${value} → ${Number(value)}` `` for any input, then run it on: "42", "42abc", "", " ", true, false, null, undefined, [] and [7]. Two of the results will surprise you — write down which two and why.